LUMEN GATEWAY / SANDBOX API v2

Bangun integrasi. Uji setiap alur. Tanpa dana nyata.

Panduan penggunaan dan integrasi untuk Dynamic QRIS dan Virtual Account. Termasuk kredensial otomatis, API, webhook, contoh aplikasi, Laravel, serta deployment untuk banyak merchant.

01 / DYNAMIC QRIS	02 / VIRTUAL ACCOUNT	03 / SIGNED EVENTS
QR unik per transaksi. Nominal dan expiry ditentukan server.	VA sintetis unik, exact amount, dan sekali pakai.	HMAC, retry, replay, serta jejak delivery yang persisten.

Sandbox independen. QR mengarah ke payer simulator, bukan payload EMV QRIS tersertifikasi. VA bukan nomor yang diterbitkan bank. Tidak ada dana nyata atau koneksi ke jaringan bank/e-wallet.

VERSI 2.0.0 / 05 NOVEMBER 2026

Quick links: portal http://127.0.0.1:4179
Online docs /docs | OpenAPI /openapi.json

Isi: setup & credentials (2), QRIS (3), VA (4), status & idempotency (5), webhook (6-7), contoh aplikasi (8), Laravel (9-10), skenario (11), shared hosting (12), troubleshooting & referensi (13).

INTEGRATION GUIDE / 02

Mulai dan minta kredensial.

Registrasi otomatis membuat merchant workspace beserta kredensial test. Tidak ada verifikasi email atau approval administrator.

Jalankan service lokal

```
cd payment-gateway-simulator
npm ci
npm start
# Open http://127.0.0.1:4179
```

Buka Minta kredensial. Masukkan nama workspace, email contoh, dan password 10-128 karakter. Registrasi mengeluarkan API key dan webhook signing secret; simpan keduanya saat dialog pertama tampil.

Nilai	Penggunaan
sk_test_...	Secret API key untuk Authorization: Bearer dari backend. Ditampilkan sekali.
whsec_test_...	Signing secret untuk verifikasi raw webhook. Terpisah dari API key.
sess_...	Sesi workspace 24 jam untuk portal management; browser memakai cookie HttpOnly.

Simpan pada environment backend

```
LUMEN_GATEWAY_URL=http://127.0.0.1:4179
LUMEN_GATEWAY_KEY=sk_test_REPLACE_ME
LUMEN_WEBHOOK_SECRET=whsec_test_REPLACE_ME
```

Melalui Kredensial API, terbitkan key per aplikasi dan cabut key yang tidak dipakai. Maksimal 10 key aktif per merchant. Key hilang harus diganti; password memakai salted scrypt dan key API disimpan sebagai hash.

Webhook secret tetap tersimpan privat untuk signing. Rotasi langsung memengaruhi semua delivery berikutnya, termasuk replay. Update receiver setelah rotasi. Email hanya identifier login, belum diverifikasi; reset password/email belum tersedia.

INTEGRATION GUIDE / 03

Dynamic QRIS.

Satu transaksi menghasilkan satu QR unik, satu nominal yang terkunci, dan batas waktu pembayaran.

Buat dari backend (bash/curl)

```
curl -X POST http://127.0.0.1:4179/v1/payments \
  -H "Authorization: Bearer YOUR_TEST_SECRET_KEY" \
  -H "Idempotency-Key: invoice-101-deposit-1" \
  -H "Content-Type: application/json" \
  -d '{"external_id":"INV-101-DP-1","amount":150000,
       "payment_method":"qris","expires_in":900,
       "description":"Booking deposit"}'
```

Respons inti (201 Created; field lain juga tersedia)

```
{
  "id": "pay_example",
  "external_id": "INV-101-DP-1",
  "amount": 150000, "currency": "IDR", "status": "pending",
  "payment_method": "qris",
  "checkout_url": "http://127.0.0.1:4179/pay/cs_test_...",
  "qr_string": ".../qris/cs_test_...?amount=150000",
  "qr_image_url": ".../checkout-api/cs_test_.../qr.svg",
  "simulation_only": true
}
```

Buka checkout_url lalu pindai QR dengan kamera ponsel. URL GET /qris/{token} langsung mencatat pembayaran uji dan menampilkan konfirmasi. Nominal mengikuti data server; scan ulang tidak menggandakan receipt. Event payment.paid dikirim ke aplikasi secara terpisah.

Amount	Expiry	Single use
Rp1 - Rp10.000.000, integer IDR	60-3600 detik; default 900	Setelah paid/expired, QR tidak aktif lagi.

qr_string adalah URL payer sandbox, bukan string EMV QRIS. QR tidak bekerja pada aplikasi pembayaran bank/e-wallet. Untuk Wi-Fi lokal, set SCAN_HOST ke IPv4 komputer dan SCAN_BASE_URL=http://IP:4181. Listener terpisah hanya membuka /qris/*; portal tetap localhost. Untuk pengguna jarak jauh gunakan HTTPS.

INTEGRATION GUIDE / 04

Virtual account.

VA memakai nominal tertutup (closed amount), nomor unik per transaksi, dan referensi bank untuk mendeteksi duplikasi.

```
curl -X POST http://127.0.0.1:4179/v1/payments \
  -H "Authorization: Bearer YOUR_TEST_SECRET_KEY" \
  -H "Idempotency-Key: invoice-102-deposit-1" \
  -H "Content-Type: application/json" \
  -d '{"external_id":"INV-102-DP-1","amount":250000,
       "payment_method":"virtual_account","bank_code":"BCA",
       "expires_in":86400}'
```

Field	Perilaku
bank_code	BCA, BNI, BRI, MANDIRI, PERMATA
account_number	Nomor sintetis unik 16 digit berawalan 999; tidak diterbitkan bank.
amount	Rp1 - Rp100.000.000, integer IDR. Kurang/lebih ditolak.
expires_in	60-604800 detik; default 86400 (24 jam).
checkout_url	Instruksi VA dan tombol Buka bank simulator.

Uji penerimaan bank

1. Buat VA dari dashboard atau API, lalu buka checkout.
2. Pilih Buka bank simulator.
3. Masukkan nominal yang sama persis dan referensi bank uji.
4. Konfirmasi. Provider menjadi paid; webhook mengonfirmasi aplikasi.
5. Coba nominal salah untuk melihat AMOUNT_MISMATCH tanpa receipt.

> Referensi yang sama pada payment yang sama tidak menambah receipt kedua. Referensi yang sudah dipakai pada payment lain ditolak. Reference baru juga tidak dapat mengkredit payment yang sudah paid. Jangan transfer ke VA sintetis menggunakan bank nyata.

Nomor tidak digunakan ulang saat instrumen expired atau cancelled. Buat payment baru dengan external_id baru untuk percobaan berikutnya.

INTEGRATION GUIDE / 05

Status, retry, dan idempotency.

Simpan payment ID dan idempotency key pada aplikasi. Timeout request tidak berarti pembayaran gagal dibuat.

Endpoint	Tujuan
GET /v1/payment-methods	Kemampuan dan limit channel.
POST /v1/payments	Buat QRIS / VA. Wajib Idempotency-Key.
GET /v1/payments/{id}	Status provider yang otoritatif.
GET /v1/payments	Filter external_id, limit 1-100, cursor starting_after.
POST /v1/payments/{id}/cancel	Tutup payment pending. Repeating cancel aman.
POST /v1/payments/{id}/refunds	Buat refund pending. Wajib Idempotency-Key.
GET /v1/refunds/{id}	Pantau status refund.

Gunakan key yang sama (8-100 karakter) dan body identik ketika mengulang request setelah timeout. Body berbeda dengan key sama menghasilkan 409 IDEMPOTENCY_CONFLICT. Scope key adalah merchant + operasi; catatan disimpan bersama persistent state.

external_id juga unik per merchant. Sertakan identitas invoice, tahap DP/pelunasan, dan nomor attempt. Gunakan external_id baru setelah instrumen lama ditutup. Idempotent replay mengembalikan resource yang sama dengan status terkini.

```
pending -> paid | failed | expired | cancelled
paid -> refund.pending -> refund.succeeded | refund.failed
expired + late receipt -> payment.late_received (review)
```

Return URL hanya navigasi. Jangan menandai invoice lunas dari query string/browser. Cocokkan merchant_id, payment ID, external_id, amount, currency=IDR, serta status dari backend. Paid tidak dapat diturunkan oleh event pending/failed yang terlambat.

Provider paid dan webhook delivered adalah dua fakta berbeda. Invoice aplikasi harus mengikuti konfirmasi backend yang terverifikasi. Event lama adalah snapshot historis; GET status dapat lebih baru.

INTEGRATION GUIDE / 06

Kontrak webhook.

Konfigurasikan receiver pada Webhook & event sebelum membuat payment. Gateway akan mengirim event secara asynchronous.

Headers dan signature

```
X-Simulator-Event-Id: evt_example
X-Simulator-Timestamp: <Unix seconds at delivery>
X-Simulator-Signature: <64 lowercase hex characters>

HMAC_SHA256(webhook_secret, timestamp + "." + exact_raw_body)
```

Periksa timestamp dengan toleransi +/-300 detik, bandingkan signature secara constant time, lalu parse JSON. payload.id wajib sama dengan header event ID. Gunakan webhook secret, bukan API key.

Contoh payment.paid

```
{
  "id": "evt_example", "type": "payment.paid",
  "api_version": "2026-11-04",
  "created_at": "2026-11-04T05:00:00.000Z",
  "data": {
    "payment_id": "pay_example", "merchant_id": "mer_example",
    "external_id": "INV-101-DP-1", "amount": 150000,
    "currency": "IDR", "payment_method": "qris",
    "bank_code": null, "status": "paid",
    "paid_amount": 150000, "refunded_amount": 0
  }
}
```

Event types	Tindakan aplikasi
payment.pending / failed / expired / cancelled	Update status yang sesuai; jangan downgrade invoice paid.
payment.paid	Verifikasi invoice + GET status; catat receipt sekali.
payment.late_received	Rekonsiliasi. Tidak membuka pesanan.
refund.pending / succeeded / failed	Update ledger refund terpisah. Ada refund_id/refund_amount.

Payment menyimpan snapshot endpoint. Event tanpa URL berstatus unconfigured; menyimpan URL akan mengantrekannya. Replay dapat memilih snapshot lama atau URL merchant terbaru.

INTEGRATION GUIDE / 07

Retry dan pemrosesan event.

Delivery bersifat at least once. Receiver dapat menerima ID event yang sama beberapa kali, termasuk setelah restart.

Attempt	Jeda setelah kegagalan sebelumnya
Initial	Saat event tersedia untuk worker
Retry 1 / 2	5 detik / 30 detik
Retry 3 / 4	120 detik / 600 detik
Retry 5 / 6	1800 detik / 7200 detik
Setelah habis	Status failed; manual replay memulai siklus baru.

Respons 2xx mengakui event. Network error, timeout 4 detik, redirects, dan non-2xx memicu retry. Worker setiap detik memproses maksimal 10 event due per batch. Pending retry dan expiry kembali diproses setelah service aktif lagi.

Payload dan event ID tetap sama; timestamp dan signature dibuat ulang pada setiap delivery. Snapshot event tetap dapat dikirim walaupun status provider sudah lebih baru. Manual replay tidak boleh menggandakan receipt di aplikasi.

Pola receiver yang disarankan

1. Verifikasi raw signature, timestamp, dan event ID.
2. Cocokkan payment/invoice/merchant/amount/currency.
3. Ambil status provider dari backend saat memproses payment.paid.
4. Dalam satu transaksi database aplikasi, simpan event ID unik, receipt unik, dan perubahan invoice.
5. Acknowledge duplicate dengan 2xx tanpa perubahan saldo kedua.

> Jika pekerjaan bisnis terlalu lama, durably enqueue event yang sudah valid sebelum respons 2xx, lalu proses di worker aplikasi. Mengembalikan 2xx sebelum event tersimpan dapat kehilangan notifikasi. Contoh Node berikut memakai memory hanya untuk demo; integrasi produksi perlu dedupe yang durable.

Signature memakai raw bytes. Jangan JSON.parse lalu JSON.stringify untuk menghitung ulang HMAC: whitespace atau urutan properti akan berubah. Gunakan body persis seperti yang diterima.

INTEGRATION GUIDE / 08

Jalankan contoh aplikasi.

Contoh Node mandiri memperlihatkan alur create payment, hosted checkout, signature verification, dan pembaruan invoice melalui webhook.

Siapkan environment (.env di folder simulator)

```
GATEWAY_BASE_URL=http://127.0.0.1:4179
GATEWAY_API_KEY=sk_test_REPLACE_ME
GATEWAY_WEBHOOK_SECRET=whsec_test_REPLACE_ME
EXAMPLE_PORT=4181


# Terminal 1: gateway
npm start
# Terminal 2: app contoh (tidak mencetak secret)
node --env-file=.env examples/merchant-app.mjs
```

Pada portal, simpan webhook URL http://127.0.0.1:4181/webhooks/lumen. Local HTTP callback harus diizinkan di konfigurasi development gateway.

Buat transaksi melalui aplikasi contoh

```
curl -X POST http://127.0.0.1:4181/demo/payments \
  -H "Content-Type: application/json" \
  -d '{"external_id":"DEMO-QR-1","amount":150000,
       "payment_method":"qris"}'

# Buka payment.checkout_url dari respons, lalu bayar uji.
curl http://127.0.0.1:4181/demo/invoices
```

Invoice contoh berubah menjadi paid hanya setelah receiver memverifikasi HMAC dan provider status. Untuk VA, kirim payment_method=virtual_account dan bank_code=BCA. Uji juga webhook gagal dan replay melalui portal.

File examples/merchant-app.mjs adalah runnable tanpa Laravel. Invoice dan processed event ID pada contoh bersifat in-memory; restart contoh menghapusnya. Gateway utama menyimpan state persisten. Jangan memakai contoh memory ini sebagai penyimpanan invoice produksi.

INTEGRATION GUIDE / 09

Hubungkan dari Laravel.

Ini titik integrasi untuk aplikasi Laravel yang akan dibangun. Tidak ada project Laravel, migration, atau database yang dibuat oleh simulator.

Tambahkan ke config/services.php

```
'lumen' => [
    'url' => env('LUMEN_GATEWAY_URL'),
    'key' => env('LUMEN_GATEWAY_KEY'),
    'webhook_secret' => env('LUMEN_WEBHOOK_SECRET'),
],
```

Create payment dari backend yang sudah mengotorisasi invoice

```
use Illuminate\Support\Facades\Http;

// Simpan key per attempt, reuse ketika request timeout.
$key = $invoice->gateway_idempotency_key;
$payment = Http::acceptJson()
    ->withToken(config('services.lumen.key'))
    ->withHeaders(['Idempotency-Key' => $key])
    ->connectTimeout(3)->timeout(10)
    ->post(config('services.lumen.url').'/v1/payments', [
        'external_id' => $invoice->number.'-DP-'.$attemptNumber,
        'amount' => (int) $invoice->deposit_due,
        'payment_method' => 'qris',
        'expires_in' => 900,
    ])->throw()->json();

// Simpan payment ID + external ID pada attempt sebelum redirect.
return redirect()->away($payment['checkout_url']);
```

Field invoice di atas adalah contoh untuk disesuaikan dengan schema aplikasimu. Hitung amount di backend dari invoice, bukan dari request browser. Gunakan key yang sama dan body identik untuk retry transport.

Untuk VA: payment_method=virtual_account, bank_code=BCA, expires_in=86400. DP dan pelunasan adalah payment berbeda, masing-masing dengan external_id dan idempotency key sendiri.

> Laravel snippets lengkap: examples/laravel-integration.md. Referensi metode HTTP client memakai API Laravel 12.x; sesuaikan routing/middleware dengan versi aplikasi yang dipilih.

INTEGRATION GUIDE / 10

Receiver Laravel.

Endpoint webhook menerima POST tanpa login session. Lindungi melalui signature, timestamp, invoice matching, dan deduplikasi.

Verifikasi sebelum memproses event

```
$raw = $request->getContent();
$timestamp = (string) $request->header(
    'X-Simulator-Timestamp', '');
$signature = (string) $request->header(
    'X-Simulator-Signature', '');

abort_unless(ctype_digit($timestamp), 401);
abort_if(abs(time() - (int) $timestamp) > 300, 401);
$expected = hash_hmac('sha256', $timestamp.'.'.$raw,
    config('services.lumen.webhook_secret'));
abort_unless(hash_equals($expected, $signature), 401);

$event = json_decode($raw, true, 512, JSON_THROW_ON_ERROR);
abort_unless(isset($event['id'], $event['data']) &&
    $event['id'] === $request->header(
        'X-Simulator-Event-Id'), 422);
```

Schema-dependent processing (adaptasi diperlukan)

Gunakan endpoint API stateless, misalnya /api/webhooks/lumen. Jika memakai web middleware, exempt CSRF hanya pada route callback bertanda tangan ini. Jangan menonaktifkan CSRF secara global.

Cari dan lock invoice/attempt berdasarkan payment ID + external ID. Pada payment.paid, lakukan GET provider status dan bandingkan merchant ID, amount integer, IDR, serta status. Terapkan receipt/invoice dan processed event ID unik dalam satu transaksi. Acknowledge duplicate dengan 2xx.

> Pseudofield invoice dan instruksi transaksi bukan controller siap pakai atau migration. Sesuaikan dengan schema app. Jangan insert event ID lebih dahulu lalu gagal memperbarui invoice; retry berikutnya dapat diabaikan secara salah.

Refund dan late_received diproses terpisah. Return URL tidak membuat invoice paid. Respons receiver harus cepat; gunakan durable queue bila proses melampaui timeout delivery.

INTEGRATION GUIDE / 11

Uji hasil dan pengecualian.

Gunakan dashboard detail transaksi atau hosted payer untuk menilai alur aplikasi sebelum integrasi pembayaran nyata.

Skenario	Ekspektasi
Nominal VA salah	422 AMOUNT_MISMATCH, tanpa receipt atau payment.paid.
Referensi bank duplikat	Tidak ada receipt kedua; paid tidak diturunkan.
Batas waktu berakhir	Satu payment.expired; QR/VA ditutup.
Dana setelah expiry	Merchant Uji dana terlambat: needs_review, status tetap terminal.
Receiver 500 / timeout	Provider tetap paid; delivery retry sesuai jadwal.
Replay event	ID tetap; aplikasi dedupe dan acknowledge 2xx.
Cabut API key	Request berikut ditolak 401; transaksi lama tetap ada.
Refund parsial	Pending mereservasi saldo; succeeded menambah refunded_amount sekali.
Refund gagal	Bisa dicoba ulang bila jumlah masih tersedia.
Login merchant lain	Tidak bisa membaca/mengubah transaksi merchant pertama.

Refund

```
POST /v1/payments/pay_example/refunds
Authorization: Bearer YOUR_TEST_SECRET_KEY
Idempotency-Key: refund-invoice-101-partial-1

{"amount":50000,"reason":"Partial booking cancellation"}
```

Refund berstatus pending, lalu simulasikan succeeded/failed melalui portal. Total pending+succeeded tidak boleh melebihi dana paid. Late/quarantined funds tidak memakai endpoint refund settled payment ini; lakukan rekonsiliasi operator di luar demo.

INTEGRATION GUIDE / 12

Shared sandbox untuk banyak user.

Setiap merchant memperoleh workspace dan kredensial sendiri. Deployment memerlukan HTTPS, persistent disk, dan satu proses server.

```
HOST=0.0.0.0
PORT=4179
PUBLIC_BASE_URL=https://sandbox.your-domain.example
ALLOW_LOCAL_WEBHOOKS=false
DATA_FILE=/persistent/lumen/gateway.json

node --env-file=.env server.mjs
```

Gunakan reverse proxy TLS ke Node service. Forward seluruh route portal, v1, pay, issuer, docs, dan asset. Public binding ditolak bila PUBLIC_BASE_URL bukan HTTPS atau local webhook masih diaktifkan.

State default berada di .data/gateway.json. Akun, hash key, pembayaran, idempotency, dan retry queue disimpan melalui serialized atomic replacement. Backup file ini dan batasi akses: signing secret dan checkout token juga tersimpan di dalamnya.

> Satu proses / satu instance. Jangan menjalankan cluster, beberapa replica, atau dua proses yang menulis file yang sama. Persistent disk wajib; ephemeral/serverless filesystem tidak cocok. Untuk beban lebih besar, ganti file store dengan transactional database dan queue terpisah.

Kontrol default	Nilai
IP requests	300 request/menit/IP
Merchant / API bucket	120 request/menit per bucket
Login / signup	10 login/menit/IP; 5 pendaftaran/jam/IP
JSON / credential	64 KB body; maksimal 10 API key aktif per akun
Public callbacks	HTTPS port 443, private/reserved IP dan redirects diblokir.

Server tidak mempercayai forwarded IP headers. Di belakang proxy, terapkan edge rate controls per user/IP; built-in IP limits melihat koneksi proxy. DNS callback divalidasi setiap delivery dan alamat valid dipin ke koneksi untuk membatasi rebinding.

Registrasi tetap otomatis tanpa approval. Hosting tidak otomatis dipublikasikan oleh perubahan repository ini. Tidak ada live keys, real fund transfer, email delivery, atau reset password.

INTEGRATION GUIDE / 13

Troubleshooting dan referensi.

API.md dan OpenAPI JSON adalah kontrak lengkap dari implementasi simulator. Panduan ini menjelaskan penggunaan dan keputusan integrasi.

Gejala	Periksa
401 INVALID_API_KEY	Gunakan sk_test_, bukan sess_ / webhook secret; pastikan key belum dicabut.
409 IDEMPOTENCY_CONFLICT	Body atau nilai key berubah. Reuse body identik untuk retry.
409 REFERENCE_EXISTS	external_id pernah dipakai. Retrieve payment lama atau gunakan reference attempt baru.
422 AMOUNT_MISMATCH	Nominal harus persis integer IDR sesuai payment.
Webhook pending/failed	Endpoint, signature secret, status respons, timeout 4s, dan DNS/public URL rules.
QR ponsel tidak membuka page	URL 127.0.0.1 hanya lokal perangkat. Pakai sandbox HTTPS yang reachable.
Data hilang setelah deploy	Pastikan DATA_FILE berada pada persistent disk dan tidak dihapus saat update.
Email/password hilang	Email tidak diverifikasi; password reset belum tersedia. Simpan akses test workspace.

Referensi perilaku dan API (diakses 04 November 2026)

Bank Indonesia - QRIS dynamic amount and limit
https://www.bi.go.id/id/fungsi-utama/sistem-pembayaran/ritel/kanal-layanan/qris/default.aspx

Midtrans - notification verification and idempotency
https://docs.midtrans.com/docs/https-notification-webhooks

Xendit - webhook handling and retries
https://docs.xendit.co/docs/handling-webhooks

Xendit - API credential separation
https://docs.xendit.co/docs/api-keys

Laravel 12.x - HTTP client API
https://api.laravel.com/docs/12.x/Illuminate/Http/Client/PendingRequest.html

QRIS test cap Rp10 juta mengikuti penjelasan Bank Indonesia. VA limits, expiry ranges, HMAC protocol, dan retry intervals adalah kebijakan simulator sendiri. Tidak ada klaim sertifikasi QRIS atau kompatibilitas SDK provider tertentu.